

Configuring HTTPS servers

[HTTPS server optimization](#)

[SSL certificate chains](#)

[A single HTTP/HTTPS server](#)

[Name-based HTTPS servers](#)

[An SSL certificate with several names](#)

[Server Name Indication](#)

[Compatibility](#)

To configure an HTTPS server, the `ssl` parameter must be enabled on [listening sockets](#) in the [server](#) block, and the locations of the [server certificate](#) and [private key](#) files should be specified:

```
server {  
    listen          443 ssl;  
    server_name     www.example.com;  
    ssl_certificate  www.example.com.crt;  
    ssl_certificate_key www.example.com.key;  
    ssl_protocols   TLSv1.2 TLSv1.3;  
    ssl_ciphers     HIGH:!aNULL:!MD5;  
    ...  
}
```

The server certificate is a public entity. It is sent to every client that connects to the server. The private key is a secure entity and should be stored in a file with restricted access, however, it must be readable by nginx' s master process. The private key may alternately be stored in the same file as the certificate:

```
ssl_certificate      www.example.com.cert;  
ssl_certificate_key  www.example.com.cert;
```

in which case the file access rights should also be restricted. Although the certificate and the key are stored in one file, only the certificate is sent to a client.

The directives [ssl_protocols](#) and [ssl_ciphers](#) can be used to limit connections to include only the strong versions and ciphers of SSL/TLS. By default nginx uses “`ssl_protocols TLSv1.2 TLSv1.3`” and “`ssl_ciphers HIGH:!aNULL:!MD5`”, so configuring them explicitly is generally not needed. Note that default values of these directives were [changed](#) several times.



NGINX
SPONSORED BY F5

[english](#)

[русский](#)

[news](#)

[about](#)

[download](#)

[security](#)

[documentation](#)

[faq](#)

[books](#)

[community](#)

[enterprise](#)

[community forum](#)

[\(new\)](#)

[x.com](#)

[blog](#)

[njs](#)

[ingress controller](#)

[gateway fabric](#)

HTTPS server optimization

SSL operations consume extra CPU resources. On multi-processor systems several [worker processes](#) should be run, no less than the number of available CPU cores. The most CPU-intensive operation is the SSL handshake. There are two ways to minimize the number of these operations per client: the first is by enabling [keepalive](#) connections to send several requests via one connection and the second is to reuse SSL session parameters to avoid SSL handshakes for parallel and subsequent connections. The sessions are stored in an SSL session cache shared between workers and configured by the [ssl_session_cache](#) directive. One megabyte of the cache contains about 4000 sessions. The default cache timeout is 5 minutes. It can be increased by using the [ssl_session_timeout](#) directive. Here is a sample configuration optimized for a multi-core system with 10 megabyte shared session cache:

```
worker_processes auto;

http {
    ssl_session_cache    shared:SSL:10m;
    ssl_session_timeout  10m;

    server {
        listen            443 ssl;
        server_name        www.example.com;
        keepalive_timeout  70;

        ssl_certificate     www.example.com.crt;
        ssl_certificate_key www.example.com.key;
        ssl_protocols       TLSv1.2 TLSv1.3;
        ssl_ciphers          HIGH:!aNULL:!MD5;
        ...
    }
}
```

SSL certificate chains

Some browsers may complain about a certificate signed by a well-known certificate authority, while other browsers may accept the certificate without issues. This occurs because the issuing authority has signed the server certificate using an intermediate certificate that is not present in the certificate base of well-known trusted certificate authorities which is distributed with a particular browser. In this case the authority provides a bundle of chained certificates which should be concatenated to the signed server certificate. The server certificate must appear before the chained certificates in the combined file:

```
$ cat www.example.com.crt bundle.crt > www.example.com.chained
```

The resulting file should be used in the [ssl_certificate](#) directive:

```
server {  
    listen          443 ssl;  
    server_name     www.example.com;  
    ssl_certificate  www.example.com.chained.crt;  
    ssl_certificate_key www.example.com.key;  
    ...  
}
```

If the server certificate and the bundle have been concatenated in the wrong order, nginx will fail to start and will display the error message:

```
SSL_CTX_use_PrivateKey_file("/.../www.example.com.key") failed  
(SSL: error:05800074:x509 certificate routines::key values
```

because nginx has tried to use the private key with the bundle's first certificate instead of the server certificate.

Browsers usually store intermediate certificates which they receive and which are signed by trusted authorities, so actively used browsers may already have the required intermediate certificates and may not complain about a certificate sent without a chained bundle. To ensure the server sends the complete certificate chain, the `openssl` command-line utility may be used, for example:

```
$ openssl s_client -connect www.godaddy.com:443  
...  
Certificate chain  
 0 s:/C=US/ST=Arizona/L=Scottsdale/1.3.6.1.4.1.311.60.2.1.3=U  
   /1.3.6.1.4.1.311.60.2.1.2=AZ/O=GoDaddy.com, Inc  
   /OU=MIS Department/CN=www.GoDaddy.com  
   /serialNumber=0796928-7/2.5.4.15=V1.0, Clause 5. (b)  
  i:/C=US/ST=Arizona/L=Scottsdale/O=GoDaddy.com, Inc.  
   /OU=http://certificates.godaddy.com/repository  
   /CN=Go Daddy Secure Certification Authority  
   /serialNumber=07969287  
 1 s:/C=US/ST=Arizona/L=Scottsdale/O=GoDaddy.com, Inc.  
   /OU=http://certificates.godaddy.com/repository  
   /CN=Go Daddy Secure Certification Authority  
   /serialNumber=07969287  
  i:/C=US/O=The Go Daddy Group, Inc.  
   /OU=Go Daddy Class 2 Certification Authority  
 2 s:/C=US/O=The Go Daddy Group, Inc.  
   /OU=Go Daddy Class 2 Certification Authority  
  i:/L=ValiCert Validation Network/O=ValiCert, Inc.  
   /OU=ValiCert Class 2 Policy Validation Authority
```

```
/CN=http://www.valicert.com/emailAddress=info@valicert.
```

In this example the subject (“s”) of the `www.GoDaddy.com` server certificate #0 is signed by an issuer (“i”) which itself is the subject of the certificate #1, which is signed by an issuer which itself is the subject of the certificate #2, which is signed by the well-known issuer *ValiCert, Inc.* whose certificate is stored in the browsers’ built-in certificate base (that lay in the house that Jack built).

If a certificate bundle has not been added, only the server certificate #0 will be shown.

A single HTTP/HTTPS server

It is possible to configure a single server that handles both HTTP and HTTPS requests:

```
server {
    listen      80;
    listen      443 ssl;
    server_name www.example.com;
    ssl_certificate www.example.com.crt;
    ssl_certificate_key www.example.com.key;
    ...
}
```

Prior to 0.7.14 SSL could not be enabled selectively for individual listening sockets, as shown above. SSL could only be enabled for the entire server using the [ssl](#) directive, making it impossible to set up a single HTTP/HTTPS server. The `ssl` parameter of the [listen](#) directive was added to solve this issue. The use of the [ssl](#) directive in modern versions is thus discouraged; it was removed in 1.25.1.

Name-based HTTPS servers

A common issue arises when configuring two or more HTTPS servers listening on a single IP address:

```
server {
    listen      443 ssl;
    server_name www.example.com;
    ssl_certificate www.example.com.crt;
    ...
}

server {
    listen      443 ssl;
    server_name www.example.org;
```

```
ssl_certificate www.example.org.crt;  
...  
}
```

With this configuration a browser receives the default server's certificate, i.e. `www.example.com` regardless of the requested server name. This is caused by SSL protocol behaviour. The SSL connection is established before the browser sends an HTTP request and nginx does not know the name of the requested server. Therefore, it may only offer the default server's certificate.

The oldest and most robust method to resolve the issue is to assign a separate IP address for every HTTPS server:

```
server {  
    listen          192.168.1.1:443 ssl;  
    server_name     www.example.com;  
    ssl_certificate www.example.com.crt;  
    ...  
}  
  
server {  
    listen          192.168.1.2:443 ssl;  
    server_name     www.example.org;  
    ssl_certificate www.example.org.crt;  
    ...  
}
```

An SSL certificate with several names

There are other ways that allow sharing a single IP address between several HTTPS servers. However, all of them have their drawbacks. One way is to use a certificate with several names in the SubjectAltName certificate field, for example, `www.example.com` and `www.example.org`. However, the SubjectAltName field length is limited.

Another way is to use a certificate with a wildcard name, for example, `*.example.org`. A wildcard certificate secures all subdomains of the specified domain, but only on one level. This certificate matches `www.example.org`, but does not match `example.org` and `www.sub.example.org`. These two methods can also be combined. A certificate may contain exact and wildcard names in the SubjectAltName field, for example, `example.org` and `*.example.org`.

It is better to place a certificate file with several names and its private key file at the *http* level of configuration to inherit their single memory copy in all servers:

```

ssl_certificate      common.crt;
ssl_certificate_key  common.key;

server {
    listen            443 ssl;
    server_name       www.example.com;
    ...
}

server {
    listen            443 ssl;
    server_name       www.example.org;
    ...
}

```

Server Name Indication

A more generic solution for running several HTTPS servers on a single IP address is [TLS Server Name Indication extension](#) (SNI, RFC 6066), which allows a browser to pass a requested server name during the SSL handshake and, therefore, the server will know which certificate it should use for the connection. SNI is currently [supported](#) by most modern browsers and is a mandatory-to-implement extension in TLSv1.3, though may not be used by some old or special clients.

Only domain names can be passed in SNI, however some browsers may erroneously pass an IP address of the server as its name if a request includes literal IP address. One should not rely on this.

In order to use SNI in nginx, it must be supported in both the OpenSSL library with which the nginx binary has been built as well as the library to which it is being dynamically linked at run time. OpenSSL supports SNI since 0.9.8f version if it was built with config option “--enable-tlsex” . Since OpenSSL 0.9.8j this option is enabled by default. If nginx was built with SNI support, then nginx will show this when run with the “-V” switch:

```

$ nginx -V
...
TLS SNI support enabled
...

```

However, if the SNI-enabled nginx is linked dynamically to an OpenSSL library without SNI support, nginx displays the warning:

```
nginx was built with SNI support, however, now it is linked
dynamically to an OpenSSL library which has no tlsext support
therefore SNI is not available
```

Compatibility

- The SNI support status has been shown by the “-V” switch since 0.8.21 and 0.7.62.
- The `ssl` parameter of the [listen](#) directive has been supported since 0.7.14. Prior to 0.8.21 it could only be specified along with the `default` parameter.
- SNI has been supported since 0.5.23.
- The shared SSL session cache has been supported since 0.5.6.
- Version 1.27.3 and later: the default SSL protocols are TLSv1.2 and TLSv1.3 (if supported by the OpenSSL library). Otherwise, when OpenSSL 1.0.0 or older is used, the default SSL protocols are TLSv1 and TLSv1.1.
- Version 1.23.4 and later: the default SSL protocols are TLSv1, TLSv1.1, TLSv1.2, and TLSv1.3 (if supported by the OpenSSL library).
- Version 1.9.1 and later: the default SSL protocols are TLSv1, TLSv1.1, and TLSv1.2 (if supported by the OpenSSL library).
- Version 0.7.65, 0.8.19 and later: the default SSL protocols are SSLv3, TLSv1, TLSv1.1, and TLSv1.2 (if supported by the OpenSSL library).
- Version 0.7.64, 0.8.18 and earlier: the default SSL protocols are SSLv2, SSLv3, and TLSv1.
- Version 1.0.5 and later: the default SSL ciphers are “ HIGH:!aNULL:!MD5 ” .
- Version 0.7.65, 0.8.20 and later: the default SSL ciphers are “ HIGH:!ADH:!MD5 ” .
- Version 0.8.19: the default SSL ciphers are “ ALL:!ADH:RC4+RSA:+HIGH:+MEDIUM ” .
- Version 0.7.64, 0.8.18 and earlier: the default SSL ciphers are “ ALL:!ADH:RC4+RSA:+HIGH:+MEDIUM:+LOW:+SSLv2:+EXP ” .

written by Igor Sysoev
edited by Brian Mercer